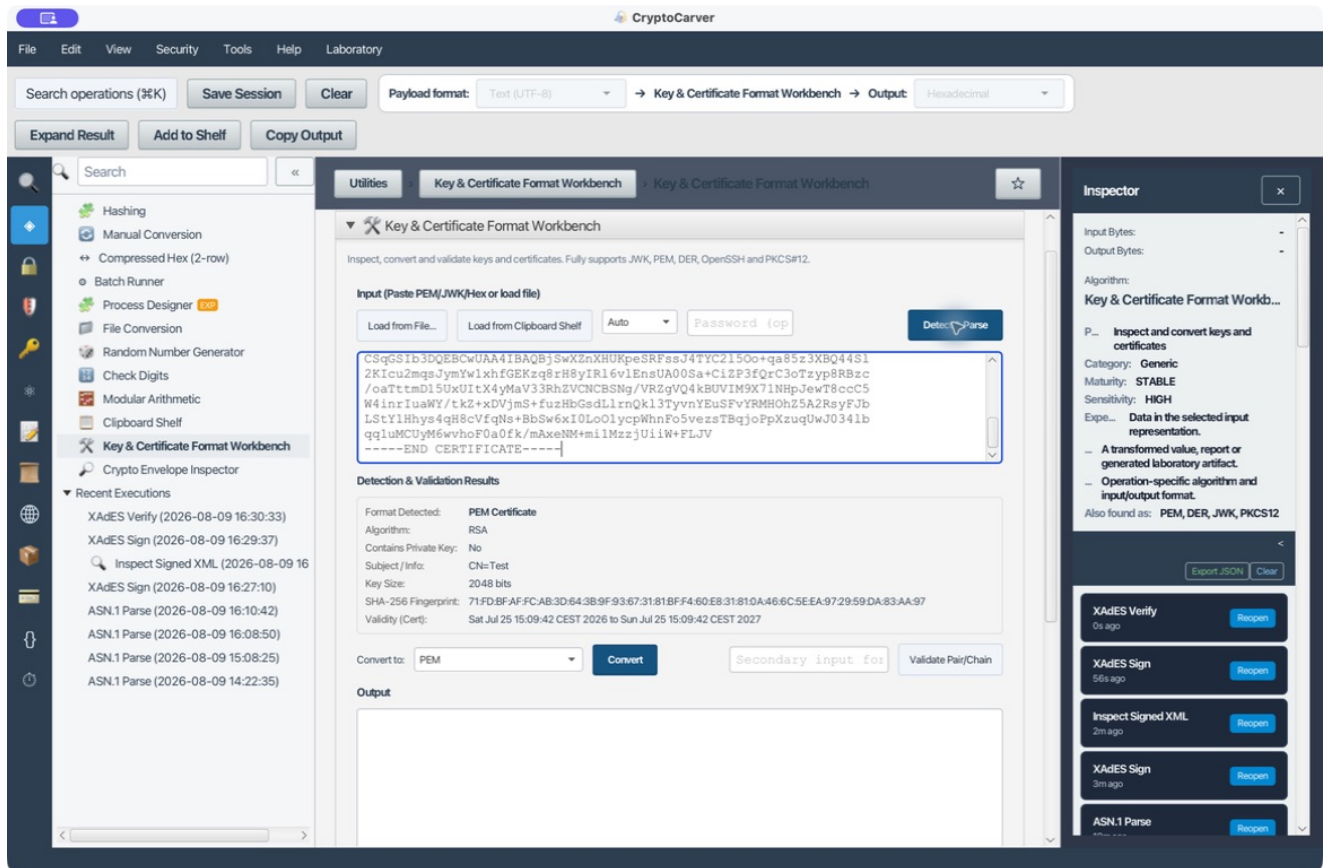


Formatos de claves, almacenes y PKCS#11



Este laboratorio responde a una pregunta habitual: «tengo una clave o un certificado, pero no sé qué representa exactamente ni cuál es el contenedor correcto para moverlo». Se parte de una inspección, se hace una conversión verificable y se termina en la frontera entre un almacén software y un token PKCS#11.

Trabaja con material de laboratorio. Un PEM o un PKCS#12 pueden contener una clave privada aunque parezcan simples ficheros de texto.

Mapa de decisión

Si necesito...	Formato o contenedor recomendable	Precaución
Transportar un certificado legible	PEM X.509	PEM es DER codificado en Base64; las cabeceras importan.
Intercambiar un certificado binario	DER	No es texto ni hexadecimal salvo que se represente así.
Entregar una clave privada interoperable	PKCS#8 PEM, cifrado cuando proceda	PKCS#1 es específico de RSA; PKCS#8 es el contenedor genérico.
Integrar una clave pública en una API web	JWK	Comprueba kty, crv/n/e, kid y el uso declarado.
Usar SSH	OpenSSH public/private key	No confundir con PEM aunque ambos sean texto.
Llevar clave y cadena con contraseña	PKCS#12 (.p12/.pfx)	La contraseña protege el contenedor, no sustituye la custodia de claves.
Operar sin exportar la privada	PKCS#11	El objeto vive en token/HSM y se referencia por slot, etiqueta o CKA_ID.

Caso 1. Quiero saber qué es un PEM recibido

Objetivo. Identificar un certificado sin abrirlo con un editor ni asumir que el bloque Base64 es una clave.

- 1 Abre Genéricas → Key & Certificate Format Workbench.
- 2 Pega este certificado de laboratorio y deja el selector en Auto:

-----BEGIN CERTIFICATE-----

```
MIICvTCCAaWgAwIBAgIEa88uhzANBgkqhkiG9w0BAQsFADAPMQ0wCwYDVQQDEwRU
ZXN0MB4XDTI2MDcyNTEzMDk0MloXDTI3MDcyNTEzMDk0MlowDzENMA5GA1UEAxME
VGZvdDCCASlWdQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBAIacb5AqpD8HvGUS
cSE+/QQZcyiHskWei94pYwllQ+P1jgi+1Zdm7yynjUdsylZXYWYXJTtSkY8eCDgb
TOBWgkh8UI56Tq/fjSr1qBxdmjjzerYlby3re0KcZjfhTVlwAObnbKURyiD1R0Xe
BYVP/Qo4bjq2G/pJBUvB69bUVkDSBSi/u4dpwT5FzVZZ0+aTizxotxy0gREyTYxg
nILYFFRwXaUJkCG5+Pyzc+pla46mg5kcrboNKniUOfR0vvYzgTs0WljX9FVBj6Qo
Vch67uBmUeVcS+Sbu987x57qlz+KRSxB/aFs5qPMtDCKX8S2BQ7PIQv+IYJoQV5U
4wmYAOECAwEAAAMhMB8wHQYDVR0OBBYEFepOS1VxqHcnTiklybS0G8JscxWNMA0G
CSqGSIb3DQEBCwUAA4IBAQBjSwXZnXHUKpeSRFssJ4TYC2l5Oo+qa85z3XBQ44SI
2Klcu2mqSjYmYw1xhfGEKzq8rH8yIRl6VIEnsUA00Sa+CiZP3fQrC3oTzyp8RBzc
/oaTttmD15UxUItX4yMaV33RhZVCNCBSNg/VRZgVQ4kBUVIM9X71NHpJewT8ccC5
W4inrluaWY/tkZ+xDVjmS+fuzHbGsdLlrmQkI3TyvnYEuSFvYRMHOhZ5A2RsyFJb
LStY1Hhys4qH8cVfqNs+BbSw6xl0LoO1ycpWhnFo5vezsTBqjoPpXzuqUwJ034lb
qq1uMCUyM6wwhoF0a0fk/mAxeNM+mi1MzzjUiiW+FLJV -----END CERTIFICATE-----
```

- 1 Pulsa Detect & Parse.

Resultado reproducido. El banco detecta PEM Certificate, algoritmo RSA, tamaño 2048 bits, Contains Private Key: No, sujeto CN=Test, la huella SHA-256 y el intervalo de validez. Esos campos permiten identificar el artefacto antes de convertirlo o incluirlo en un flujo CMS/XAdES.

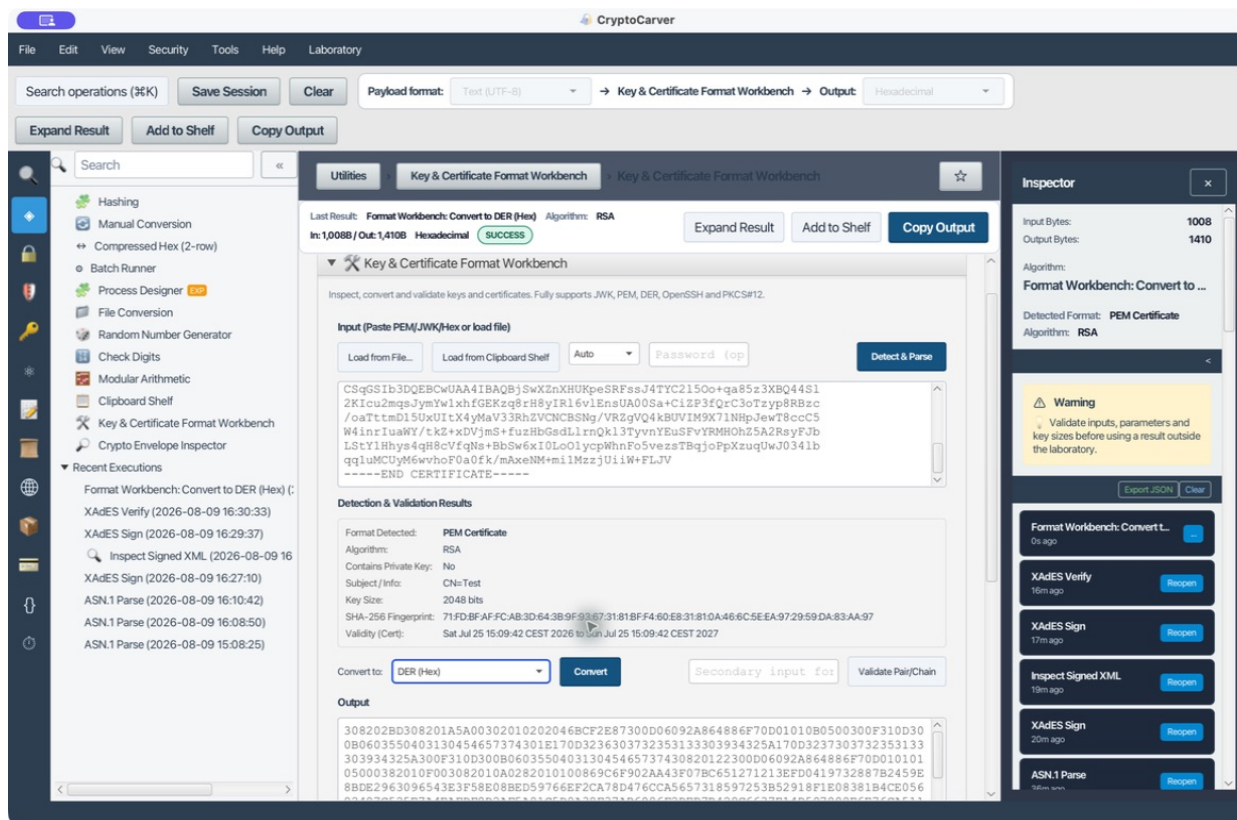
Qué comprobar. La huella es el identificador operativo: compara todos sus octetos con el valor esperado por un canal independiente. El CN es descriptivo, no prueba por sí solo la identidad ni la confianza de la cadena.

Caso 2. Quiero entregar el mismo certificado en DER hexadecimal

Situación. Un sistema de pruebas intercambia DER como hexadecimal. No hay que volver a emitir el certificado: hay que cambiar únicamente su representación.

- 1 Conserva el PEM ya analizado.
- 2 En Convert to, selecciona DER (Hex).
- 3 Pulsa Convert.

Salida reproducida. CryptoCarver informa de una conversión correcta, con entrada de 1,008 bytes y salida de 1,410 caracteres hexadecimales. La salida comienza por 308202BD...: 30 es el SEQUENCE ASN.1 exterior del certificado DER. El número de caracteres hexadecimales es el doble de los bytes DER; no es un nuevo certificado ni una clave.



Caso 2: PEM a DER hexadecimal

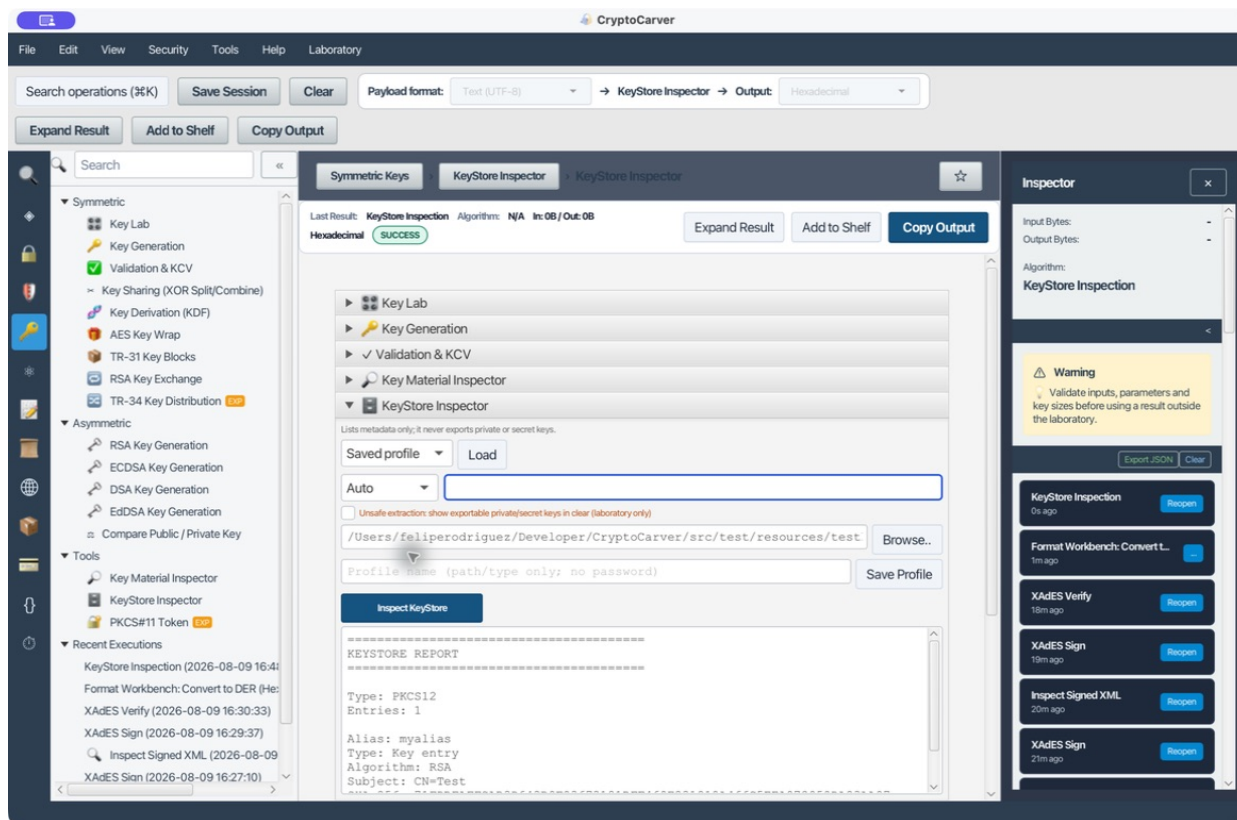
Validación de vuelta. Pega esa salida en el mismo banco, selecciona la entrada hexadecimal si no se detecta automáticamente y analiza de nuevo. Deben coincidir algoritmo, sujeto, periodo y huella SHA-256. Si cambia la huella, no se ha hecho una conversión de codificación: se ha cambiado el artefacto.

Caso 3. Quiero inventariar un PKCS#12 sin exponer material secreto

Objetivo. Ver alias, tipo de entrada, certificado y metadatos de un .p12 sin exportar la privada.

- 1 Abre Claves → KeyStore Inspector.
- 2 Tipo: Auto.
- 3 Fichero: src/test/resources/testks.p12.
- 4 Introduce la contraseña de laboratorio storepass y pulsa Inspect KeyStore.

Resultado reproducido. El informe identifica Type: PKCS12, Entries: 1, alias myalias, Type: Key entry, algoritmo RSA y Subject: CN=Test. La interfaz declara expresamente que lista metadatos y no exporta privadas ni secretas. Mantén desmarcada la casilla de extracción insegura: solo se justifica para una práctica local controlada.



Caso 3: inventario no exportable de PKCS#12

Prueba negativa útil. Sustituye temporalmente la contraseña por una incorrecta. El resultado debe ser un error de apertura, no una lista vacía interpretada como almacén sin entradas. Restablece la contraseña antes de proseguir. Nunca guardes una contraseña dentro de un perfil de CryptoCarver.

Caso 4. Quiero crear un token PKCS#11 de laboratorio

PKCS#11 no es otro formato de fichero. Es una API para tokens: la biblioteca del fabricante expone slots, cada slot puede contener un token y el token guarda objetos. Una clave privada bien configurada se usa mediante operaciones como C_Sign o C_Decrypt; no se lee como bytes.

CryptoCarver consume un token ya aprovisionado. Para crear uno reproducible se utiliza SoftHSM únicamente como HSM software de laboratorio. En macOS con Homebrew, estos son los pasos ejecutados para el token de esta guía:

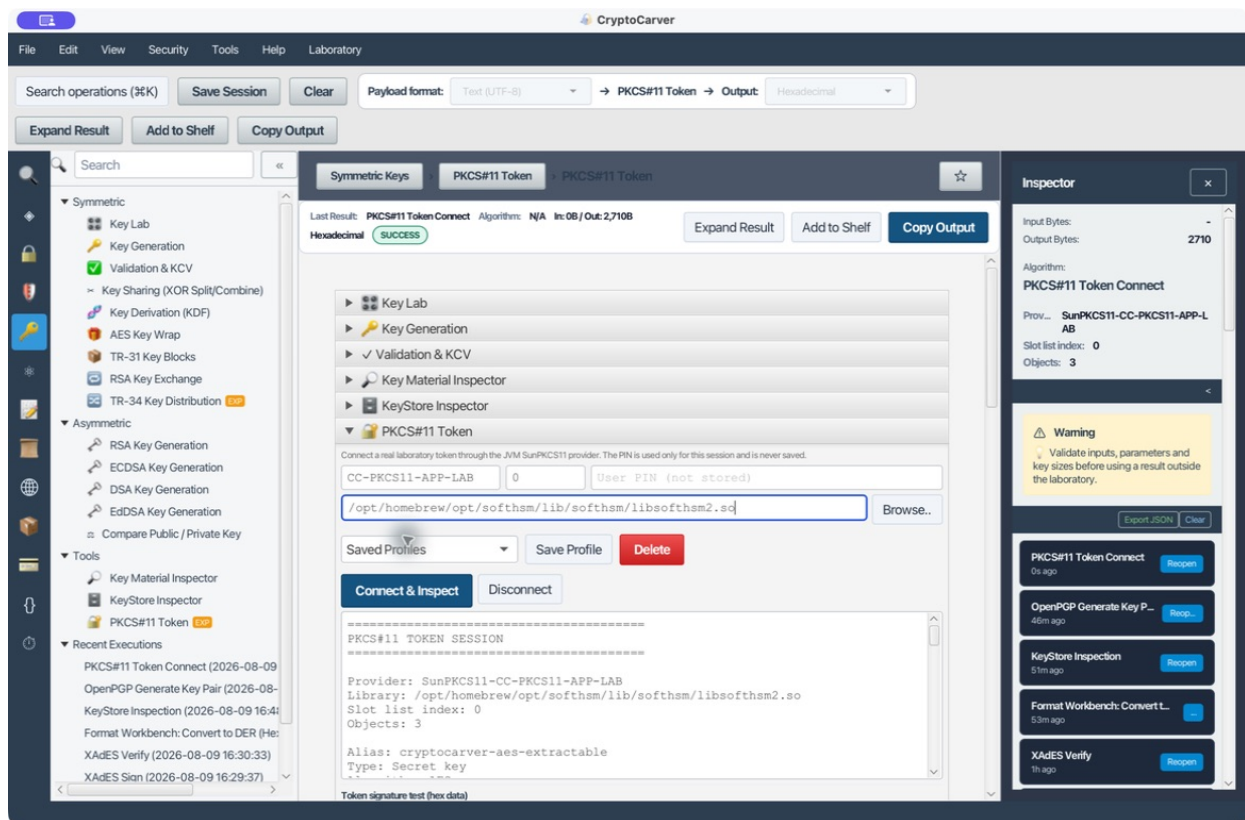
- 1 Inicializa un slot libre con la etiqueta CC-PKCS11-APP-LAB, un PIN de seguridad de laboratorio y un PIN de usuario de laboratorio.
- 2 Genera dentro del token un par RSA de 2048 bits, etiqueta cc-app-signing e identificador 02.
- 3 Conserva el resultado importante: la privada se marca sensitive, always sensitive, never extractable y local.

El primer PIN administra el token; el PIN de usuario abre una sesión de operación. Ninguno se copia a un perfil de CryptoCarver ni a una receta. En un HSM real, la ceremonia de inicialización, el SO-PIN, la política de generación y la auditoría pertenecen al procedimiento de custodia de la organización.

Caso 5. Quiero conectar CryptoCarver y obtener un inventario del token

- 1 Abre Claves → PKCS#11 Token.
- 2 Introduce nombre CC-PKCS11-APP-LAB, Slot index: 0 y la ruta del módulo libsoftsm2.
- 3 Introduce el PIN de usuario solo para la sesión actual y pulsa Connect & Inspect.

Resultado reproducido. CryptoCarver conectó el proveedor SunPKCS11-CC-PKCS11-APP-LAB, informó del slot 0 y encontró tres objetos. El informe lista alias, tipo, algoritmo y formato, pero para las claves residentes muestra Fingerprint: Not exported. También separa la compatibilidad JCA de la lista de mecanismos: que el proveedor anuncie un algoritmo no garantiza que cualquier objeto tenga sus atributos de uso.



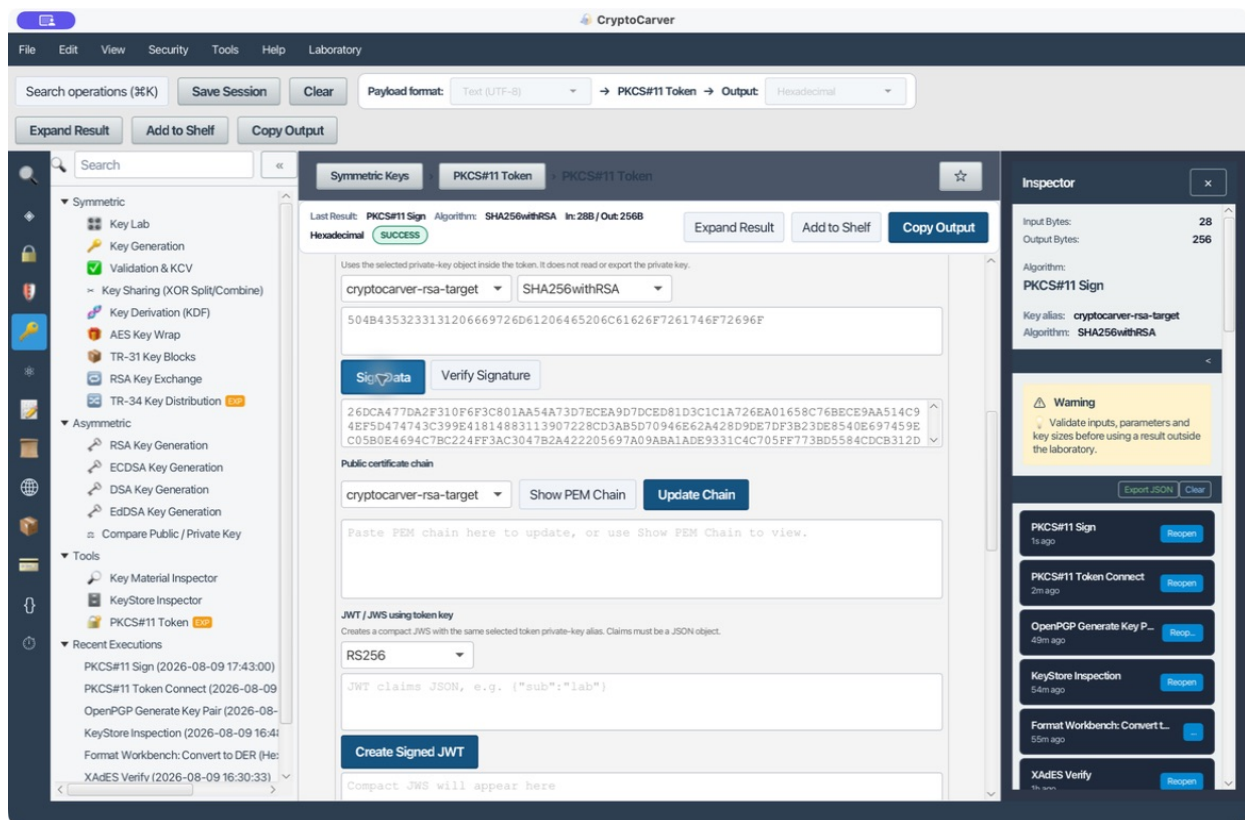
Caso 5: conexión e inventario PKCS#11

Campos que deben quedar documentados.

Campo	Ejemplo de laboratorio	Motivo
Biblioteca	/opt/homebrew/opt/softsm/lib/softsm/libsoftsm2.so	CryptoCarver carga el proveedor SunPKCS11 contra ese módulo.
slotListIndex	0	Selecciona la posición de la lista de slots, no el número hexadecimal visible en otra herramienta.
Etiqueta de token	CC-PKCS11-APP-LAB	Comprobación humana ante tokens conectados simultáneamente.
Alias / CKA_ID	cryptocarver-rsa-target / identificador del objeto	El alias facilita el uso; CKA_ID es el identificador estable de interoperabilidad.
Atributos	CKA_SIGN, CKA_DECRYPT, CKA_WRAP, CKA_EXTRACTABLE	Determinan qué puede hacer la clave y si el token permite sacarla envuelta.

Caso 6. Quiero firmar, verificar y conservar la clave dentro del token

En la misma sesión se seleccionó el alias privado cryptocarver-rsa-target, el algoritmo SHA256withRSA y la entrada hexadecimal 504B4353233131206669726D61206465206C61626F7261746F72696F (PKCS#11 firma de laboratorio). Tras Sign Data, CryptoCarver produjo una firma de 256 bytes. El panel identifica el alias y el algoritmo, pero no revela el material de clave.



Caso 6: firma con clave privada residente

Pulsa Verify Signature con la misma entrada y firma para la prueba positiva. Para la prueba negativa cambia un nibble de la entrada, por ejemplo el último 6F a 6E; la verificación debe fallar. No vuelvas a firmar el dato alterado: el objetivo es confirmar que la firma queda ligada exactamente a los bytes originales.

Caso 7. Quiero reutilizar la clave residente en JWT, CMS y certificados

Tras conectar el token, CryptoCarver mantiene el alias activo para varias operaciones. Estas son sus fronteras y entradas:

Operativa	Entrada reproducible	Resultado que se valida	Punto de control
JWS/JWT	<code>{"sub":"laboratorio","scope":"firmar","iat":1760000000}</code>	JWS compacto header.payload.d.signature con RS256	Verifica con la clave pública o certificado del token.
CMS SignedData	504B435323313120434D53206465206C61626F7261746F72696F	CMS Base64 attached o detached	Inspecciona con el tutorial CMS y verifica contenido/firma nte.
Cadena pública	Alias de certificado y Show PEM Chain	PEM de certificado y cadena, nunca privada	Comprueba sujeto, emisor y huella antes de usarla.
CSR	Alias activo de PKCS#11 y datos de sujeto	CSR cuya prueba de posesión se firma en token	Valida que la clave pública de CSR corresponde al objeto seleccionado.
XAdES / PAdES / ASiC	Fuente Connected PKCS#11 Token	Firma de documento/XML con la misma clave residente	Valida firma y política de confianza por separado.

Para CMS detached, conserva el contenido original exactamente: el CMS no lo incluye, por lo que el verificador necesita ambos artefactos. Para JWT, iat es una política de aplicación; no convierte por sí solo

un token en válido para siempre ni reemplaza la validación de iss, aud, expiración y algoritmo permitido.

Caso 8. Quiero envolver o recuperar una clave sin enseñarla a la JVM

La sección Wrap / Unwrap key objects permite que un objeto de clave sea procesado dentro del token. Selecciona una clave pública de envoltura, una clave destino y una transformación compatible; el resultado será material envuelto hexadecimal. Para recuperar, selecciona la parte privada correspondiente, pega el blob, indica algoritmo y tipo del objeto de salida.

La política manda sobre la interfaz: si el objeto destino no tiene CKA_EXTRACTABLE=true, una operación de envoltura puede ser rechazada. Ese rechazo es una protección correcta, no un error a sortear. En producción se recomienda crear la clave destino directamente en el token y usar una KEK o mecanismo de transporte aprobado, en lugar de habilitar exportación para «hacer que funcione».

Diagnóstico ordenado si no conecta. (1) comprueba que la ruta carga la biblioteca correcta y su arquitectura; (2) confirma slotListIndex y la etiqueta; (3) verifica PIN y estado de login; (4) busca el CKA_ID exacto; (5) revisa mecanismos y atributos antes de diseñar el flujo. No intentes resolver una ausencia de mecanismo exportando la clave del HSM.

Errores que evitan incidentes

Síntoma	Causa probable	Acción segura
-----BEGIN RSA PRIVATE KEY----- no funciona donde se espera PKCS#8	PKCS#1 frente a PKCS#8	Convierte explícitamente y vuelve a analizar.
Un PEM se interpreta como texto base64	Se han perdido cabeceras o saltos de línea	Restaura BEGIN/END; no recortes el contenido.
Certificado y privada no validan como par	Son artefactos de orígenes distintos	Usa Validate Pair/Chain y compara la clave pública derivada.
PKCS#12 abre pero no hay cadena esperada	Falta certificado intermedio	Inventaría aliases y entrega la cadena requerida, no solo el leaf.
PKCS#11 lista el token pero la firma falla	Mecanismo, atributos de uso o login incorrectos	Revisa mecanismo y CKA_SIGN; no cambies políticas de producción para probar.

Checklist antes de integrar

- He identificado el tipo real mediante análisis, no solo por extensión.
- He comparado huella o clave pública tras cualquier conversión.
- Sé si el objetivo requiere clave pública, certificado, cadena o clave privada PKCS#8.
- El PKCS#12 se inventaría sin extracción insegura.
- Para PKCS#11, conozco biblioteca, slot, etiqueta, CKA_ID, mecanismos y política de no exportación.
- Las contraseñas y PIN no aparecen en capturas, perfiles ni recetas exportadas.